OPENCV • PYTHON

Travaux pratiques – Initiation au traitement d’images

Fiche étudiant | Niveau débutant | Durée indicative : 2 h à 2 h 30

Nom : ..... Groupe : ..... Date : .....

Objectifs pédagogiques

• Installer et importer OpenCV avec Python.

• Charger, afficher et enregistrer une image.

• Convertir une image en niveaux de gris et la redimensionner.

• Appliquer un flou et détecter les contours.

• Réaliser une première détection de visages avec un classificateur pré-entraîné.

Préparation de l'environnement

• Installer Python 3 et un éditeur (VS Code, PyCharm ou IDLE).

• Créer un dossier de travail nommé TP_OpenCV.

• Placer dans ce dossier une image nommée image.jpg et, pour le dernier exercice, une photo contenant un ou plusieurs visages nommée personnes.jpg.

• Dans le terminal, exécuter : pip install opencv-python

Rappel : vérifier l'installation

• Créer un fichier test_opencv.py et saisir le code ci-dessous.

```
import cv2
print("Version OpenCV :", cv2.__version__)
```

TP 1 — Lire et afficher une image

• Consigne : charger image.jpg, vérifier que le chargement a réussi, puis afficher l'image dans une fenêtre.

• Questions : à quoi sert imread() ? Pourquoi utilise-t-on waitKey(0) ? Que se passe-t-il si le nom du fichier est incorrect ?

```
import cv2

image = cv2.imread("image.jpg")

if image is None:
    print("Erreur : image.jpg introuvable")
else:
    cv2.imshow("Mon image", image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 2 — Convertir en niveaux de gris

• Consigne : convertir l'image en niveaux de gris et afficher côte à côte (dans deux fenêtres) l'image originale et l'image transformée.

• Question : quelle différence observez-vous entre les deux images ?

```
import cv2

image = cv2.imread("image.jpg")
if image is None:
    print("Image introuvable")
else:
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Originale", image)
    cv2.imshow("Niveaux de gris", gris)
```

```
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 3 — Redimensionner

• Consigne : créer une version de l'image de largeur 400 pixels et de hauteur 300 pixels. Afficher les deux versions.

• Question : dans cv2.resize(image, (400, 300)), quel nombre représente la largeur ?

```
import cv2

image = cv2.imread("image.jpg")
if image is not None:
    petite = cv2.resize(image, (400, 300))
    cv2.imshow("Originale", image)
    cv2.imshow("Redimensionnee", petite)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 4 — Flouter une image

• Consigne : appliquer GaussianBlur() avec une fenêtre (7, 7), puis afficher le résultat.

• Question : quel est l'effet du flou sur les détails de l'image ?

```
import cv2

image = cv2.imread("image.jpg")
if image is not None:
    floue = cv2.GaussianBlur(image, (7, 7), 0)
    cv2.imshow("Originale", image)
    cv2.imshow("Image floue", floue)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 5 — Détecter les contours

• Consigne : convertir l'image en niveaux de gris, puis appliquer Canny avec les seuils 100 et 200. Afficher le résultat.

• Question : que représentent les lignes blanches dans l'image obtenue ?

```
import cv2

image = cv2.imread("image.jpg")
if image is not None:
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    contours = cv2.Canny(gris, 100, 200)
    cv2.imshow("Contours", contours)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 6 — Mini-défi : détecter les visages

• Consigne : utiliser le fichier haarcascade_frontalface_default.xml fourni par OpenCV. Détecter les visages dans personnes.jpg et dessiner un rectangle vert autour de chaque visage.

• À rendre : le fichier Python, une capture d'écran du résultat et trois phrases expliquant le fonctionnement général.

```
import cv2

detecteur = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml"
)

image = cv2.imread("personnes.jpg")
if image is None:
    print("Image introuvable")
elif detecteur.empty():
    print("Fichier XML du détecteur introuvable")
else:
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    visages = detecteur.detectMultiScale(
        gris, scaleFactor=1.1, minNeighbors=5
    )

    for (x, y, w, h) in visages:
        cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

```
print("Nombre de visages :", len(visages))
cv2.imshow("Detection de visages", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Bilan et livrable

• Remettre les fichiers .py, les images de résultats et une courte réponse aux questions.

• Bonus : enregistrer l'image transformée avec cv2.imwrite('resultat.jpg', image).

Espace de réponses

1. Que retourne imread() si le fichier n'existe pas ?

.....

.....

2. Quelle est la différence entre BGR et niveaux de gris ?

.....

.....

3. Quel est l'effet de GaussianBlur() ?

.....

.....

4. Dans quel type de situation la détection de visages peut-elle échouer ?

.....

.....